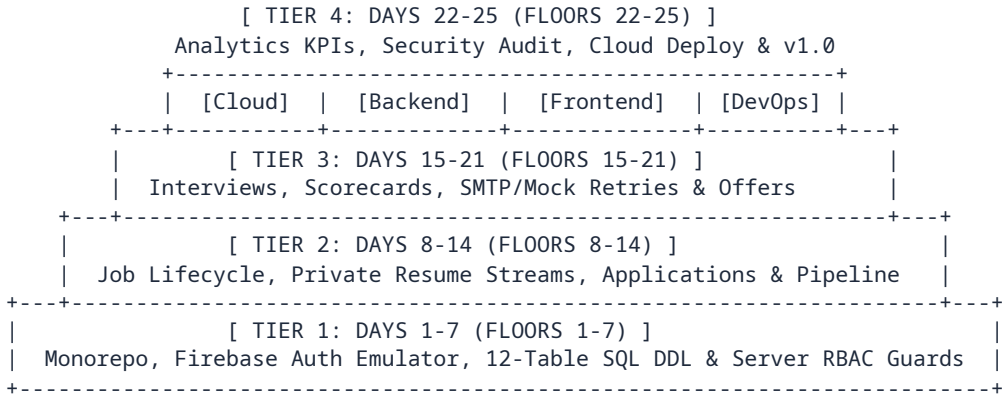
Project 1: Smart Recruitment & Applicant Tracking System (ATS)

3.5-Week (25-Day) Pyramid Team Execution & Daily Git Commit Checklist
Synchronized 4-Role Engineering Schedule: Frontend Developer • Backend Developer • DevOps Engineer • Cloud Engineer

How the Pyramid Floor Execution Model Works

To finish the complete Smart ATS in exactly **3.5 weeks (25 working days)** while maintaining a clean, incremental Git history where **every team member creates 1 runnable Git commit per day (100 total commits)**, this plan structures development like building a **25-floor pyramid**:

- **4 Colored Blocks per Floor (Intra-Day Handoff Chain)**: Every single day represents one horizontal floor of the pyramid (`Floor 1` to `Floor 25`). Each floor is constructed from four distinct blocks laid in a logical dependency order (`Block A → Block B → Block C → Block D`) by the **Cloud Engineer**, **Backend Developer**, **Frontend Developer**, and **DevOps Engineer**.
- **Zero Idle Blocking**: Within each day, Block A establishes the cloud/database or API contract so Block B and Block C can immediately wire the server logic and React interface, while Block D locks the floor in place with automated tests, CI checks, or container configs.
- **Floor-by-Floor Vertical Progression**: Floor *N + 1* never starts until all 4 blocks of Floor *N* are committed, verified, and merged into `main`.



Team Role (Block Color)	Primary Domain Ownership Across All 25 Floors	Total Commits
Cloud Engineer *(Sapphire Block)*	Google Firebase Auth & Local Emulator (`firebase.json`), `firebase-admin` setup, Custom Claims, PostgreSQL tuning, private resume storage streams, SMTP/mock config, and cloud deployment.	**25 Commits** (1/day, Days 1–25)
Backend Developer *(Emerald Block)*	12-table PostgreSQL DDL migrations, Express TypeScript APIs, JWT/RBAC middleware, transactional pipeline state machine, resume magic-byte validation, offer engine, and SQL analytics.	**25 Commits** (1/day, Days 1–25)
Frontend Developer *(Amber Block)*	React + TypeScript SPA, Firebase Client SDK (`AuthContext`), Axios Bearer interceptor, all 7 role-scoped dashboards, Kanban board, modals, and accessibility/responsive polish.	**25 Commits** (1/day, Days 1–25)

Team Role (Block Color)	Primary Domain Ownership Across All 25 Floors	Total Commits
DevOps Engineer *(Crimson Block)*	Monorepo Git governance, `docker-compose`, SQL migration automation, GitHub Actions CI gates, Jest/Supertest RBAC & security suites, Playwright E2E tests, and production Dockerfiles.	**25 Commits** (1/day, Days 1–25)

Total Duration: 3.5 Weeks (25 Days) | **Daily Output:** 4 Coordinated Commits / Floor | **Grand Total:** 100 Verified Git Commits

Tier 1 (Week 1): Foundation, Firebase Identity & Core Data Model

Floor 1 (Day 1 of 25): Monorepo Initialization, Cloud Project Setup & Base App Skeletons

Floor Objective & Handoff Chain: Establish the repository structure, Firebase cloud project, Express API shell, and React SPA shell.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** DevOps (Block A)	**DevOps Engineer**	Initialize Git monorepo (`frontend/`, `backend/`, `database/`, `tests/`, `docs/`, `uploads/`), `.gitignore`, and branch protection rules.	`chore(repo): initialize monorepo structure and gitignore rules`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Create Google Firebase project, enable Email/Password & Google OAuth providers, and add `firebase.json` for local Auth Emulator on port `9099`.	`feat(cloud): configure firebase project and local auth emulator`
☐ **3.** Backend (Block C)	**Backend Developer**	Scaffold Node.js + Express + TypeScript server (`backend/src/app.ts`) with `/api/health` endpoint, CORS, Helmet, and `.env.example`.	`feat(backend): scaffold express typescript server and health route`
☐ **4.** Frontend (Block D)	**Frontend Developer**	Scaffold React + Vite + TypeScript app (`frontend/src/App.tsx`) with base routing shell, CSS design tokens, and `.env.example`.	`feat(frontend): initialize react vite typescript shell and base router`

Floor 2 (Day 2 of 25): Database Provisioning, Core User/Company Schema & Linting Pipeline

Floor Objective & Handoff Chain: Provision PostgreSQL, execute initial user/company DDL migrations, and enforce code quality checks in CI.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Cloud (Block A)	**Cloud Engineer**	Configure PostgreSQL instance parameters, connection pooling settings, and secret environment templates for local and cloud environments.	`feat(cloud): add postgresql connection config and secret templates`
☐ **2.** Backend (Block B)	**Backend Developer**	Write `database/migrations/001_users_companies.sql` (`user_role` ENUM, `users` with `firebase_uid` `UNIQUE`, `companies`) and DB pool (`database.ts`).	`feat(db): create users and companies schema migration and db pool`

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **3.** DevOps (Block C)	**DevOps Engineer**	Create `docker-compose.yml` spinning up PostgreSQL + Firebase Emulator, plus ESLint/Prettier check workflow in `.github/workflows/lint.yml`.	`ci(devops): add docker-compose postgres setup and eslint workflow`
☐ **4.** Frontend (Block D)	**Frontend Developer**	Build responsive application shell (`Navbar`, `Sidebar`, `Footer`, and role-aware layout wrapper) with navigation links.	`feat(ui): build responsive app layout shell and navigation bar`

Floor 3 (Day 3 of 25): Jobs, Candidate Profiles & Applications Relational Schema

Floor Objective & Handoff Chain: Complete core recruitment tables, migration runner automation, and Firebase Client SDK wiring.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Write `002_jobs_candidates_applications.sql` (`jobs`, `candidate_profiles`, `applications`, `resume_metadata`, `application_stage_history`).	`feat(db): add jobs candidate_profiles applications and stage history ddl`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Configure private resume storage bucket/directory permissions and document environment variables (`UPLOAD_DIR`, `MAX_UPLOAD_BYTES`).	`feat(cloud): configure private resume storage paths and iam rules`
☐ **3.** DevOps (Block C)	**DevOps Engineer**	Create automated SQL migration runner script (`scripts/migrate.sh`) and wire it into GitHub Actions CI database service container.	`ci(db): add automated sql migration runner script to ci pipeline`
☐ **4.** Frontend (Block D)	**Frontend Developer**	Configure Firebase Web SDK (`frontend/src/config/firebaseClient.ts`) with automatic toggle for local Auth Emulator (`connectAuthEmulator`).	`feat(auth-ui): configure firebase client sdk with emulator toggle`

Floor 4 (Day 4 of 25): Interviews, Offers, Notifications DDL & Firebase Admin SDK Wiring

Floor Objective & Handoff Chain: Complete the 12-table relational schema and connect Firebase Admin SDK for server-side token verification.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Write `003_interviews_offers_notifications.sql` (`interview_events`, `interview_feedback`, `offer_records`, `onboarding_tasks`, `notification_events`).	`feat(db): add interviews feedback offers onboarding and notification tables`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Implement `backend/src/config/firebaseAdmin.ts` supporting both service-account credentials and `FIREBASE_AUTH_EMULATOR_HOST`.	`feat(cloud): initialize firebase-admin sdk with emulator and cert support`

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build Login, Registration, and Google Sign-In pages (`/login`, `/register`) with client-side form validation and error banners.	`feat(auth-ui): create login registration and google oauth screens`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Configure Jest + Supertest test harness in `backend/jest.config.ts` with isolated test database setup and teardown hooks.	`test(devops): configure jest and supertest harness for backend api`

Floor 5 (Day 5 of 25): Server-Side RBAC Middleware & Firebase-to-SQL Account Synchronization

Floor Objective & Handoff Chain: Enforce server-side JWT verification and synchronize Firebase identities with PostgreSQL user rows.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement `authenticateFirebase` and `requireRole` middlewares (`middleware/auth.ts`) plus `POST /api/auth/sync` upsert endpoint.	`feat(auth-api): implement firebase jwt middleware and sql user sync route`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Create Custom Claims sync utility (`setCustomUserClaims`) so role updates mirror across PostgreSQL `users.role` and Firebase ID tokens.	`feat(cloud): add firebase custom claims synchronization helper`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build `AuthContext.tsx` and Axios interceptor (`apiClient.ts`) that injects `Authorization: Bearer <idToken>` and syncs SQL user role.	`feat(auth-ui): implement auth context and axios bearer token interceptor`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write automated Supertest unit/integration tests verifying `401 Unauthorized` on missing/expired tokens and `200 OK` on valid sync.	`test(auth): add supertest suite for token verification and user sync`

Floor 6 (Day 6 of 25): Admin Role Management, Protected Route Guards & Synthetic Seeding

Floor Objective & Handoff Chain: Enable Admin user/role management, protect frontend routes by role, and seed synthetic demo accounts.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement Admin user management API (`GET /api/admin/users`, `PATCH /api/admin/users/:id/role`, `PUT /api/admin/company`).	`feat(admin-api): add user role management and company config endpoints`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Write synthetic Firebase Auth + PostgreSQL seed script (`database/seed/seed.ts`) provisioning Admin, HR, and Candidate demo accounts.	`feat(seed): create synthetic firebase emulator and postgresql seed script`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build `<ProtectedRoute allowedRoles={...} />` wrapper and the **Page 1: Admin Overview (`/admin`)** dashboard for role & company management.	`feat(admin-ui): build protected route guard and admin overview page`

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Automate `npm run seed` inside Docker/CI pipeline and add RBAC matrix tests confirming Candidates and HR cannot call `/api/admin/*` (`403`).	`test(rbac): add automated role matrix tests for admin endpoints`

Floor 7 (Day 7 of 25): Week 1 Milestone Verification: ER Diagram, API Spec & Full Foundation Audit

Floor Objective & Handoff Chain: Lock in Week 1 deliverables: architecture docs, permission matrix, seeded dev environment, and green CI.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Cloud (Block A)	**Cloud Engineer**	Verify Firebase Auth Emulator persistence export/import scripts and document cloud IAM & data retention assumptions in `docs/`.	`docs(cloud): add firebase emulator persistence and data retention guide`
☐ **2.** Backend (Block B)	**Backend Developer**	Add global Express error handler, Zod request validation middleware, and publish `docs/api-spec.md` & `docs/permission-matrix.md`.	`feat(backend): add global error handler zod validator and api spec docs`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Add unauthorized (`403`), not-found (`404`), and loading skeleton states across all role dashboards; commit wireframe diagrams.	`feat(ui): add error boundary 403/404 pages and loading skeletons`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Configure full Week 1 GitHub Actions CI gate (`migrate -> seed -> lint -> test -> build`) and tag release `v0.1.0-week1`.	`ci(release): finalize week 1 ci gate pipeline and tag v0.1.0-week1`

Tier 2 (Week 2): Job Posting, Candidate Applications & Pipeline

Floor 8 (Day 8 of 25): Job Creation, Editing & Lifecycle State Transitions (Draft → Published → Closed)

Floor Objective & Handoff Chain: Allow HR and Admin staff to create, edit, publish, close, and archive job openings.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement Job CRUD & lifecycle API (`POST /api/jobs`, `PUT /api/jobs/:id`, `PATCH /api/jobs/:id/status`) restricted to `ADMIN` and `HR`.	`feat(jobs-api): implement job crud and status transition endpoints`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Add PostgreSQL composite indexes for job search/filtering (`status`, `department`, `location`, `published_at`) and query performance tuning.	`perf(db): add composite indexes for job filtering and search queries`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build **Page 2: Job Management (`/hr/jobs`)** with job creation/edit modal, status filter tabs, and Publish/Close/Archive action buttons.	`feat(jobs-ui): build hr job management dashboard and create/edit modal`

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write Supertest integration tests for Job CRUD, verifying server-side validation and confirming Candidates receive `403` on job mutation.	`test(jobs): add api integration tests for job crud and role guards`

Floor 9 (Day 9 of 25): Public/Candidate Job Board, Search, Filtering & Pagination

Floor Objective & Handoff Chain: Enable candidates to browse, search, and filter published job openings with responsive pagination.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement paginated `GET /api/jobs` and `GET /api/jobs/:id` supporting keyword search, department/location filters, and applicant counts.	`feat(jobs-api): add paginated job listing search and detail endpoints`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Configure API response caching headers and CORS policies for public job board read endpoints.	`feat(cloud): configure caching headers and cors rules for job listings`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build Candidate Job Board & Job Detail view (`/jobs`, `/jobs/:id`) with debounced search, filter dropdowns, pagination, and empty states.	`feat(jobs-ui): create candidate job board search filters and detail view`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Add pagination boundary and SQL injection fuzz tests for `/api/jobs` query parameters in the CI test suite.	`test(jobs): add pagination boundary and query sanitization tests`

Floor 10 (Day 10 of 25): Candidate Profile Management & Defensive Resume Upload Engine

Floor Objective & Handoff Chain: Allow candidates to maintain their profile and securely upload validated PDF/DOCX resumes.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Cloud (Block A)	**Cloud Engineer**	Build private storage service (`storageService.ts`) that writes files outside web root with UUID names and computes SHA-256 checksums.	`feat(storage): implement private resume storage service and sha256 hashing`
☐ **2.** Backend (Block B)	**Backend Developer**	Implement `GET/PUT /api/candidates/me` and Multer resume upload (`POST /api/resumes`) validating 5 MB limit and PDF/DOCX magic bytes.	`feat(resume-api): add candidate profile api and magic-byte resume upload`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build Candidate Profile & Resume Manager section on **Page 3: Candidate Dashboard (`/candidate`)** with drag-and-drop file validation.	`feat(candidate-ui): build candidate profile editor and resume upload widget`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write security tests attempting to upload `.exe`/`.html` disguised as `.pdf` and files >5 MB, asserting `400`/`413` rejection.	`test(security): add malicious file upload and size limit rejection tests`

Floor 11 (Day 11 of 25): Validated Job Application Submission & Ownership-Protected Resume Streaming

Floor Objective & Handoff Chain: Connect job applications to candidate profiles/resumes and enforce strict access control on resume downloads.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement `POST /api/applications` (atomic insert into `applications` + initial `APPLIED` audit log) and `GET /api/applications/me`.	`feat(apps-api): implement application submission and candidate history api`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Implement authenticated stream handler for `GET /api/resumes/:id/download` verifying `ADMIN`/`HR` role or candidate file ownership.	`feat(storage): add ownership-verified private resume download stream`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build validated online Application Submission Modal on `/jobs/:id` and Application Status Tracker cards on `/candidate`.	`feat(apps-ui): build job application form modal and candidate status tracker`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write IDOR security tests verifying Candidate A cannot download Candidate B's resume (`403`) and duplicate applications return `409 Conflict`.	`test(apps): add idor resume download and duplicate application tests`

Floor 12 (Day 12 of 25): Controlled Pipeline Stage Transition Machine & Immutable Audit Trail

Floor Objective & Handoff Chain: Enforce `Applied -> Screening -> Interview -> Selected -> Offered` transitions with server-side rules and audit logging.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement transactional stage machine (`stageMachine.ts`) and `PATCH /api/applications/:id/stage` recording actor & reason in `application_stage_history`.	`feat(pipeline-api): build transactional stage state machine and audit log`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Configure database transaction isolation and row-level locking (`SELECT ... FOR UPDATE`) metrics to prevent concurrent stage race conditions.	`perf(db): configure row locking and transaction telemetry for stage updates`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build **Page 4: HR Pipeline (`/hr/pipeline`)** Kanban board and stage transition confirmation modal with audit note input.	`feat(pipeline-ui): build hr pipeline kanban board and stage transition modal`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write unit & API tests for all valid and invalid stage transitions (e.g., rejecting `APPLIED -> OFFERED` or moving out of `REJECTED`).	`test(pipeline): add exhaustive state transition rule unit and api tests`

Floor 13 (Day 13 of 25): HR Candidate Search, Multi-Criteria Filtering & Application Detail Drawer

Floor Objective & Handoff Chain: Give HR staff fast search, filtering, pagination, and a complete candidate dossier view with activity history.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement `GET /api/applications` (filter by `jobId`, `stage`, `search`, `dateRange`, pagination) and `GET /api/applications/:id/history`.	`feat(pipeline-api): add multi-filter candidate search and audit history api`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Add full-text/trigram search indexes on candidate names, skills, and job titles in PostgreSQL for fast HR pipeline queries.	`perf(db): add search indexes on candidate names skills and emails`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Add candidate search/filter bar, paginated table toggle, and slide-over Candidate Dossier Drawer (resume download + audit timeline) to `/hr/pipeline`.	`feat(pipeline-ui): add hr candidate search filters pagination and audit drawer`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Set up Playwright browser E2E test framework (`tests/e2e/`) and automate the Week 2 flow: HR publishes job → Candidate applies → HR moves stage.	`test(e2e): initialize playwright and automate job-to-pipeline e2e test`

Floor 14 (Day 14 of 25): Week 2 Milestone Verification: Pipeline Stress Testing & Transition Documentation

Floor Objective & Handoff Chain: Verify end-to-end job, resume, and pipeline workflows and document status-transition rules.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Cloud (Block A)	**Cloud Engineer**	Expand `seed.ts` to populate 15 synthetic applications across all pipeline stages with sample synthetic PDF resumes in `uploads/`.	`feat(seed): expand seed script with multi-stage applications and pdf resumes`
☐ **2.** Backend (Block B)	**Backend Developer**	Document formal status-transition rules in `docs/status-transitions.md` and refine API error codes for invalid pipeline actions.	`docs(pipeline): document formal application and job status transition rules`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Polish empty/error states, pagination controls, and keyboard accessibility on the HR Pipeline and Candidate Dashboard pages.	`fix(ui): polish empty states pagination and accessibility on pipeline views`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Run full Week 2 CI test suite (API + Playwright E2E) and tag milestone release `v0.2.0-week2`.	`ci(release): verify week 2 api and e2e test gates and tag v0.2.0-week2`

Tier 3 (Week 3): Interviews, Notifications & Offer Letters

Floor 15 (Day 15 of 25): Interview Creation, Scheduling & Conflict Detection

Floor Objective & Handoff Chain: Allow HR staff to schedule interviews with valid date/time, assigned interviewers, and conflict checks.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement `POST /api/interviews` and `GET /api/interviews` with interviewer time-slot conflict validation and automatic stage check.	`feat(interviews-api): implement interview scheduling and conflict check api`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Configure UTC timestamp normalization and meeting link generator helper (`meetingService.ts`) for scheduled interview events.	`feat(cloud): add utc timezone normalizer and meeting link generator service`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build **Page 5: Interviews (`/hr/interviews` & `/candidate/interviews`)** with calendar/list toggle and Schedule Interview modal.	`feat(interviews-ui): build interview calendar list view and scheduling modal`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write API tests verifying overlapping interview slots for the same interviewer or invalid `ends_at <= starts_at` are rejected (`400/409`).	`test(interviews): add conflict detection and timestamp validation tests`

Floor 16 (Day 16 of 25): Interview Rescheduling, Cancellation & Structured Evaluator Feedback

Floor Objective & Handoff Chain: Support interview rescheduling/cancellation tracking and confidential interviewer feedback scorecards.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement `PATCH /api/interviews/:id` (reschedule/cancel with `reschedule_count++`) and `POST/GET /api/interviews/:id/feedback` (hidden from Candidates).	`feat(interviews-api): add reschedule cancel and confidential feedback routes`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Add database audit triggers/logs capturing interview reschedule history and evaluator scorecard submission timestamps.	`feat(db): add audit logging for interview reschedules and scorecard events`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build Reschedule/Cancel modal and Structured Interview Feedback Scorecard form (1-5 ratings, strengths, concerns, decision) for HR/Admin.	`feat(interviews-ui): add reschedule modal and evaluator feedback scorecard`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write authorization tests proving Candidates can view their interview time/link but receive `403 Forbidden` when querying `/feedback`.	`test(interviews): verify candidate isolation from interviewer feedback api`

Floor 17 (Day 17 of 25): Email Notification Service, Retry Handling & Labeled Simulated Mode

Floor Objective & Handoff Chain: Trigger automated notifications on application receipt, stage changes, and interview updates with delivery logging.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Cloud (Block A)	**Cloud Engineer**	Configure SMTP / transactional email provider credentials and `EMAIL_MODE=SIMULATED|LIVE_SMTP` environment switch.	`feat(cloud): configure smtp transport and simulated email mode toggle`
☐ **2.** Backend (Block B)	**Backend Developer**	Build `notificationService.ts` with exponential backoff retry (up to 2 retries) logging every attempt into `notification_events`.	`feat(notify-api): implement email notification service with retry and db log`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build Notification Log & Delivery Status viewer in Admin/HR views showing `SENT`, `FAILED`, `retry_count`, and `SIMULATED` mode badge.	`feat(notify-ui): build notification delivery log table and mode indicator`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write unit tests mocking transient SMTP failures to verify retry counter increments and `FAILED` / `SENT` status persistence.	`test(notify): add unit tests for email retry logic and simulated mode logging`

Floor 18 (Day 18 of 25): Template-Based Offer Letter Generation & Issuance Workflow

Floor Objective & Handoff Chain: Generate offer letters from controlled templates and manage `Draft -> Issued` transitions.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Build `offerTemplateService.ts` (escaping & injecting role, salary, start date) and `POST /api/offers`, `PATCH /api/offers/:id/issue`.	`feat(offers-api): build template offer letter generator and issue endpoint`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Store approved HTML offer templates and configure PDF/HTML snapshot archiving for issued offer letters.	`feat(cloud): add controlled offer letter templates and snapshot archiving`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build HR view of **Page 6: Offers & Onboarding (`/hr/offers`)** with template composer, live offer preview, and Issue Offer button.	`feat(offers-ui): build hr offer letter composer and live template preview`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write API tests confirming offers can only be issued to `SELECTED`/`OFFERED` candidates and trigger an offer notification event.	`test(offers): add offer generation state and notification trigger tests`

Floor 19 (Day 19 of 25): Candidate Offer Acceptance/Decline & Onboarding Task Checklist

Floor Objective & Handoff Chain: Allow candidates to accept or decline issued offers without exposing other candidates' data, and track onboarding tasks.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement `POST /api/offers/:id/respond` (`ACCEPTED`/`DECLINED` by owner Candidate) auto-spawning `onboarding_tasks`, plus `PATCH /api/onboarding/:id`.	`feat(onboarding-api): add candidate offer response and onboarding task api`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Enforce strict row-level ownership checks on offer and onboarding endpoints so candidates can never enumerate other offer IDs.	`security(cloud): enforce strict candidate ownership checks on offer records`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build Candidate Offer Review card (Accept/Decline actions) and interactive Onboarding Checklist on `/candidate/offers`.	`feat(onboarding-ui): build candidate offer response portal and task checklist`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write integration tests verifying Candidate A accepting an offer sets `hired_at`, creates onboarding tasks, and blocks Candidate B (`403`).	`test(onboarding): test offer acceptance transaction and cross-candidate guard`

Floor 20 (Day 20 of 25): System-Wide Activity Log & Cross-Workflow Event Wiring

Floor Objective & Handoff Chain: Unify audit trails across applications, interviews, offers, and notifications for complete traceability.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement unified `GET /api/activity` endpoint aggregating stage transitions, interview events, offer status changes, and email dispatches.	`feat(activity-api): implement unified activity history aggregation endpoint`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Optimize SQL union/join views and pagination indexes for the system activity feed and candidate timeline.	`perf(db): optimize activity log view and timeline query indexes`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Embed real-time Activity Timeline components into the Admin Overview (`/admin`) and HR Candidate Dossier drawer.	`feat(activity-ui): embed unified activity timeline in admin and hr views`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Add Playwright E2E test covering the Week 3 lifecycle: HR schedules interview → logs feedback → issues offer → Candidate accepts.	`test(e2e): automate interview feedback offer and onboarding e2e journey`

Floor 21 (Day 21 of 25): Week 3 Milestone Verification: Integration Audit & Seed Expansion

Floor Objective & Handoff Chain: Verify all collaboration, notification, and offer workflows and update synthetic demo dataset.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Cloud (Block A)	**Cloud Engineer**	Update `seed.ts` to include sample scheduled/completed interviews, evaluator scorecards, issued/accepted offers, and onboarding tasks.	`feat(seed): add interviews feedback offers and onboarding records to seed`

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **2.** Backend (Block B)	**Backend Developer**	Audit all Week 3 endpoints for edge cases (e.g., double offer acceptance, feedback on cancelled interviews) and document mock email mode.	`fix(backend): harden edge cases on interview and offer state transitions`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Refine responsive mobile/tablet layouts for the Interview Calendar, Offer Letter preview, and Onboarding Checklist.	`style(ui): refine responsive layout for interview calendar and offer views`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Execute complete Week 1–3 regression test suite in CI and tag release `v0.3.0-week3`.	`ci(release): run week 3 regression suite and tag v0.3.0-week3`

Tier 4 (Days 22–25): Analytics, Security Hardening & Deployment

Floor 22 (Day 22 of 25): Hiring Analytics Engine: Funnel Conversion, Time-to-Hire & Offer KPIs

Floor Objective & Handoff Chain: Build accurate SQL hiring analytics with explicit denominators, date/job filters, and empty-dataset handling.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Implement `GET /api/analytics/summary` computing Application count, Stage Conversion rates, Time-to-Hire (days), Interview Reliability, and Offer Acceptance.	`feat(analytics-api): implement sql hiring kpi aggregation with filters`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Verify timezone-consistent date truncation (`AT TIME ZONE 'UTC'`) and zero-division guards (`NULLIF`) across all analytical SQL queries.	`perf(analytics): add utc date boundary normalization and nullif sql guards`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Build **Page 7: Hiring Analytics (`/hr/analytics`)** with KPI cards, stage conversion funnel bars, time-to-hire display, and Job/Date/Status filters.	`feat(analytics-ui): build hiring analytics dashboard charts and filter bar`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Write analytical accuracy tests verifying SQL KPI outputs against known synthetic cohorts, empty date ranges, and missing `responded_at` dates.	`test(analytics): verify kpi denominator accuracy and empty dataset handling`

Floor 23 (Day 23 of 25): Security Hardening: Rate Limiting, Input Sanitization & Access-Control Audit

Floor Objective & Handoff Chain: Harden authentication, authorization, upload controls, rate limits, and secrets management.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** DevOps (Block A)	**DevOps Engineer**	Add `express-rate-limit` on auth/upload routes, strict CSP/Helmet headers, and automated `npm audit` + secret-scanning check in CI.	`security(devops): add api rate limiting helmet csp headers and secret scan`

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Audit Firebase token revocation handling (`checkRevoked = true`), rotate demo environment templates, and verify zero secrets in Git history.	`security(cloud): enable firebase token revocation check and audit env files`
☐ **3.** Backend (Block C)	**Backend Developer**	Audit every controller for strict SQL parameterization, XSS HTML escaping in offer templates, and consistent `401/403/404/422` error payloads.	`security(backend): enforce strict html escaping and input validation rules`
☐ **4.** Frontend (Block D)	**Frontend Developer**	Add session-expiry auto-logout handling, file-type/size client pre-checks, and clear KPI formula tooltips on the Analytics dashboard.	`feat(ui): add session expiry handling and kpi formula explanation tooltips`

Floor 24 (Day 24 of 25): Production Containerization, Cloud Deployment & Full E2E Regression

Floor Objective & Handoff Chain: Deploy the full-stack application (or finalize reproducible Docker production stack) and generate the test evidence report.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** DevOps (Block A)	**DevOps Engineer**	Create multi-stage production `Dockerfiles` for frontend (Nginx) and backend (Node Alpine) plus production `docker-compose.prod.yml`.	`build(docker): add multi-stage production dockerfiles and compose stack`
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Deploy backend API, PostgreSQL database, and React frontend to cloud hosting (Render/Railway/Cloud Run + Firebase Hosting) and seed demo data.	`deploy(cloud): configure cloud deployment pipeline and seed live demo`
☐ **3.** Backend (Block C)	**Backend Developer**	Add `/api/health/readiness` checking PostgreSQL + Firebase Admin connectivity and export Postman/OpenAPI collection in `docs/`.	`feat(backend): add readiness probe endpoint and openapi collection export`
☐ **4.** Frontend (Block D)	**Frontend Developer**	Configure production build optimization, environment-variable validation, and capture high-resolution screenshots of all 7 role dashboards.	`chore(frontend): optimize production bundle and capture dashboard screenshots`

Floor 25 (Day 25 of 25): Final Documentation, Engineering Report, Presentation & Release v1.0.0

Floor Objective & Handoff Chain: Complete README, final engineering report, test evidence package, and tag the final submission release.

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **1.** Backend (Block A)	**Backend Developer**	Write `docs/final-report.md` detailing architecture decisions, RBAC enforcement, KPI mathematical definitions, known gaps, and future improvements.	`docs(report): author final engineering design and security controls report`

Check & Seq	Team Member Role	Concrete Engineering Task & Dependency Handoff	Mandatory Daily Git Commit Message
☐ **2.** Cloud (Block B)	**Cloud Engineer**	Document complete local emulator vs. cloud deployment instructions and `.env.example` reference tables in `README.md`.	`docs(readme): document firebase setup cloud deployment and env configuration`
☐ **3.** Frontend (Block C)	**Frontend Developer**	Finalize technical presentation deck outline (`docs/presentation-walkthrough.md`) with demo credentials for Admin, HR, and Candidate journeys.	`docs(demo): add role-by-role demo walkthrough and presentation guide`
☐ **4.** DevOps (Block D)	**DevOps Engineer**	Export full Jest/Supertest + Playwright test coverage report (`docs/test-evidence.md`), verify clean `main` branch, and tag `v1.0.0`.	`ci(release): publish final test coverage evidence and tag release v1.0.0`

5. Daily Git Discipline & Pyramid Floor Merge Protocol

To satisfy the specification's **Version Control and Work Discipline** requirement and guarantee that the `main` branch stays runnable at the end of every single day, all four members follow this daily 4-step Git workflow:

1. **Branch Naming Convention**: Every morning on Day *N*, each member checks out a short-lived feature branch named `floor-N/<role>-<short-topic>` (for example: `floor-05/backend-jwt-middleware` or `floor-05/frontend-auth-context`).
2. **Sequential Block Handoff**:
   - **Block A (First Commit of the Day)** merges first after passing local lint/build checks, unlocking the schema, cloud config, or API contract needed by the rest of the floor.
   - **Block B & Block C (Middle Commits)** pull the updated `main` branch, complete the backend/cloud implementation and frontend UI integration, and open their Pull Requests.
   - **Block D (Final Commit of the Day — Floor Seal)** runs the automated unit, integration, or E2E verification suite against the combined floor changes and merges the final commit to seal Floor *N*.
3. **Strict Secret & PII Hygiene**: Never commit `.env`, Firebase service-account JSON files, or real candidate resumes (`uploads/` must remain git-ignored). Only commit `.env.example` and synthetic seed scripts.

Complete 3.5-Week (25-Day) Milestone & Commit Audit Table

Pyramid Tier	Days / Floors	Completed System Capabilities Unlocked	Commits & Release Tag
Tier 1 (Week 1)	Days 1 – 7 (Floors 1 – 7)	Monorepo, Docker + Firebase Auth Emulator, 12-table PostgreSQL DDL, `firebase-admin` RBAC middleware, `AuthContext`, Admin Overview page, and synthetic seed data.	28 Commits (7 per member) Tag: `v0.1.0-week1`
Tier 2 (Week 2)	Days 8 – 14 (Floors 8 – 14)	Job CRUD & publishing, public job search/filtering, candidate profile & magic-byte resume upload, private resume streaming, controlled stage machine, and HR Kanban pipeline.	28 Commits (7 per member) Tag: `v0.2.0-week2`
Tier 3 (Week 3)	Days 15 – 21 (Floors 15 – 21)	Interview scheduling & conflict checks, reschedule/cancel flows, confidential evaluator scorecards, SMTP/simulated email retry logger, template offer letters, and onboarding tasks.	28 Commits (7 per member) Tag: `v0.3.0-week3`

Pyramid Tier	Days / Floors	Completed System Capabilities Unlocked	Commits & Release Tag
Tier 4 (Final 0.5 Wk)	Days 22 – 25 (Floors 22 – 25)	Hiring Analytics SQL dashboard, rate limiting & security hardening, multi-stage production Dockerfiles, cloud deployment, E2E test evidence report, and final documentation.	16 Commits (4 per member) Tag: `v1.0.0`
TOTAL	**25 Days**	**100% Complete Smart Recruitment & Applicant Tracking System**	**100 Atomic Commits** (25 per member)

Final Pre-Submission Sign-Off Checklist (Day 25 Team Verification)

- ☐ **Cloud Engineer Sign-Off**: Verified `firebase.json` Auth Emulator runs offline on port `9099`, `database/seed/seed.ts` provisions synthetic Admin, HR, and Candidate users with custom claims, private resume downloads stream securely with zero public bucket URLs, and `.env.example` files contain zero real secrets.
- ☐ **Backend Developer Sign-Off**: Verified all 12 PostgreSQL tables and status enums migrate cleanly, `authenticateFirebase` and `requireRole` enforce permissions on every protected route, pipeline stage transitions follow strict state rules inside ACID transactions, and SQL analytics queries use documented denominators.
- ☐ **Frontend Developer Sign-Off**: Verified all 7 role-scoped dashboards (Admin Overview, Job Management, Candidate Dashboard, HR Pipeline, Interviews, Offers & Onboarding, and Hiring Analytics) handle loading, empty, and `403/404` error states gracefully across desktop and mobile viewports.
- ☐ **DevOps Engineer Sign-Off**: Verified all 100 atomic commits (25 per member across Floors 1–25) are merged cleanly into `main`, GitHub Actions CI passes lint, Jest/Supertest RBAC tests, and Playwright E2E journeys, and release tag `v1.0.0` is published with test coverage evidence.